

Performance Testing

Date	30 March 2026
Team ID	SWTID-2026-5841
Project Name	FitBuddy – AI Fitness Plan Generator
Maximum Marks	5 Marks

Performance Testing

Performance testing evaluates how the system behaves under expected and peak load conditions. The sections below document the testing approach, tool used, and results actually obtained by running a live load test against the FitBuddy FastAPI application.

Step 1: Testing Overview

Field	Details
Testing Tool Used	Locust (Python load-testing framework)
Type of Testing	Load Testing with increasing concurrent virtual users (10 → 25 → 50)
Target Module / API	POST /generate-workout (workout plan generation, DB write) and GET / (home page)
Test Environment	Local sandboxed Linux container, single CPU core, FastAPI app served via Uvicorn on SQLite
Test Date	30 March 2026

Note: The Gemini AI calls inside the workout/tip generator were stubbed with realistic-length placeholder text for this test. Repeatedly calling the live Gemini API under load isn't practical (cost, rate limits, and latency dominated by Google's servers rather than FitBuddy's own code), so this test isolates and measures the performance of FitBuddy's own FastAPI + SQLite layer.

Step 2: Test Scenarios

S.No	Test Scenario / Description	No. of Virtual Users	Duration (sec)	Expected Outcome
1	Light load: mixed traffic generating workout plans and loading the home page	10	10	All requests succeed, average response time well under 1 second
2	Moderate load: same mixed traffic at higher concurrency	25	10	All requests succeed, response time stays low with no errors
3	Heavy load: peak concurrency to find the app's stress point	50	10	Throughput increases with load; some latency growth acceptable, but 0% errors

4	Sustained mixed traffic: 75% plan generation (DB write) + 25% home page (static read)	10–50 (ramped)	30 total	Consistent behavior across all three stages above
---	---	----------------	----------	---

Step 3: Performance Test Results

Results below are from the heaviest stage (50 concurrent users, 10 seconds, 1,499 total requests):

S.No	Metric	Target Value	Actual Value	Status	Remarks
1	Response Time (Avg)	< 2 seconds	7 ms	Pass	Well within target at all load levels
2	Response Time (Max)	< 5 seconds	124 ms	Pass	Worst single request, still far under target
3	Throughput (Req/sec)	–	149.6 req/s	–	Scaled from 32.8 req/s (10 users) to 149.6 req/s (50 users)
4	Error Rate	< 1%	0%	Pass	0 failed requests across all 2,585 requests in the full run
5	CPU Utilization	< 80%	Not profiled per-process	N/A	Sandboxed test container exposes only 1 CPU core; see note below
6	Memory Utilization	< 80%	~12.5% of container RAM	Pass	Container memory used rose from 285 MB to 502 MB of ~4 GB total

Results across all three load stages:

Virtual Users	Total Requests	Req/sec	Avg (ms)	p95 (ms)	p99 (ms)	Max (ms)	Errors
10	328	32.8	4	8	29	33	0%
25	758	75.7	5	13	31	66	0%
50	1,499	149.6	7	25	65	124	0%

Step 4: Observations & Analysis

Key Findings:

FitBuddy's FastAPI backend handled up to 50 concurrent users with a 0% error rate across all three load stages, sustaining nearly 150 requests per second on a single CPU core. Average response time stayed in single-digit milliseconds even at peak load, with the 95th percentile staying under 25 ms.

Bottlenecks Identified:

Tail latency grows with concurrency: the 99th percentile rose from 29 ms (10 users) to 65 ms (50 users), and the single worst request reached 124 ms at peak load. This points to SQLite's single-writer lock and the single Uvicorn worker process as the limiting factors under heavier concurrent write traffic, not the AI generation step (which was stubbed out for this test). In real production use, the actual Gemini API call would add its own latency (typically 1–4 seconds) on top of this, and would dominate overall response time far more than the FastAPI/SQLite layer measured here.

Optimization Steps Taken / Recommended:

The app already includes automatic retry with exponential backoff for transient Gemini errors (429/5xx), so brief AI-side slowdowns don't fail user requests. For further scaling, running multiple Uvicorn workers behind a reverse proxy, and moving from SQLite to a server-based database such as PostgreSQL, would remove the single-writer bottleneck seen above under heavier concurrent load.

Step 5: Screenshots / Evidence

Raw console output from the Locust load test is included below in place of a screenshot, since the test was run headless from the command line:

Type	Name	# reqs	# fails	Avg	Min	Max	Med	req/s		
GET	/	366	0(0%)	2	1	56	2	36.6		
POST	/generate-workout	1133	0(0%)	9	3	124	6	113.1		
Aggregated		1499	0(0%)	7	1	124	5	149.6		
Response time percentiles (ms):		50%	66%	75%	80%	90%	95%	98%	99%	100%
Aggregated		5	6	8	9	15	25	49	65	124